1 GPU时间与transformer架构计算量分析

1.1 GPU时间的介绍

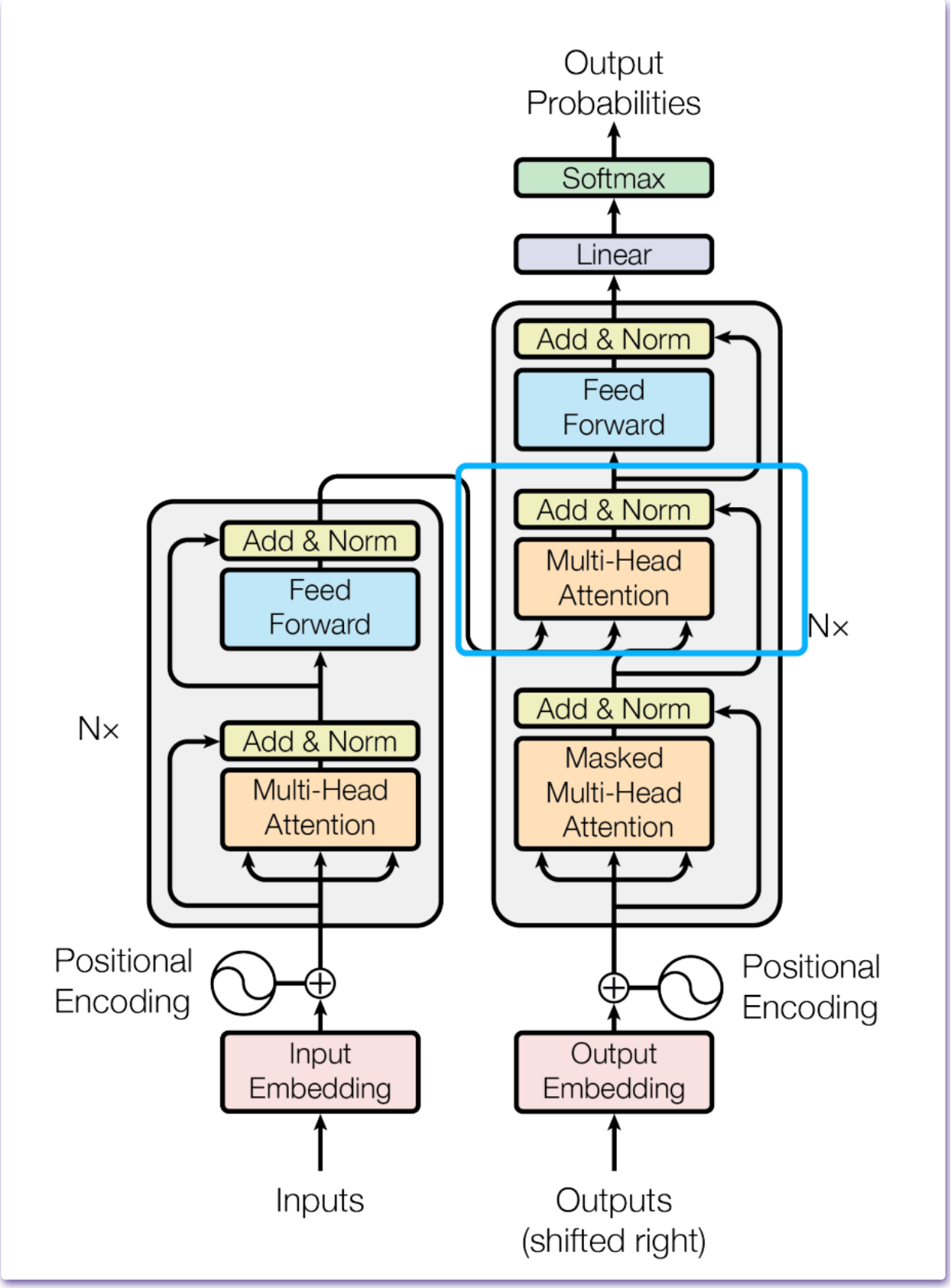
- GPU时间是指在深度学习训练和推理过程中，GPU执行计算任务所花费的时间。与CPU相比，GPU具有大量的并行计算单元，能够高效地处理矩阵运算，因此在深度学习领域被广泛应用。

  GPU时间主要包括以下几个方面：

  1. **计算时间**：执行数学运算的时间
  2. **内存访问时间**：数据在GPU内存中的读写时间
  3. **数据传输时间**：CPU与GPU之间的数据传输时间
  4. **同步时间**：不同操作之间的同步等待时间
- 影响GPU时间的因素包括：
  1. 模型的参数量和计算复杂度
  2. GPU的计算能力（如CUDA核心数量、频率等）
  3. 内存带宽
  4. 批处理大小（Batch Size）
  5. 优化器选择
  6. 并行化程度

1.2 Transformer模型的参数计算

- 这里主要讨论Decode-Only架构，所以下面的都是基于Decoder模块，且不包含encoder-decoder attention,即不包含Decoder中蓝色框的部分


Output
Probabilities
Softmax
Linear
Add & Norm
Feed
Forward
Add & Norm
Multi-Head
Attention
Add & Norm
Masked
Multi-Head
Attention
Nx
Add & Norm
Feed
Forward
Add & Norm
Multi-Head
Attention
Nx
Positional
Encoding
Positional
Encoding
Input
Embedding
Output
Embedding
Inputs
Outputs
(shifted right)

- 后续的缩写如下

	解释	缩写
$batch_size$	batch大小	b
seq_len	序列长度	s
$vocab_size$	词汇表大小	v
d_model	模型维度	d
layer_num	transformer模块个数	l

1.2.1 Embedding层参数

- Embedding层将输入的token ID映射为连续的向量表示，其参数量取决于词汇表大小和嵌入维度,假设词汇表大小为vocab_size, 模型维度为d_model,则embedding层参数量为：

$$\begin{aligned} \theta_embed &= vocab_size \cdot d_model \\ &= v \cdot d \end{aligned} \tag{1}$$

1.2.2 MultiHead-Self-Attention模型参数

- Self-Attention模块的参数主要来自于三个投影矩阵：Query (Q)、Key (K)和Value (V)，以及输出投影矩阵。

  假设输入维度为 d_model，注意力头数为 n_heads ，每个头的维度为d_k（通常d_k = d_model/n_heads）：

  Q、K、V投影矩阵以及输出投影矩阵共四个，单个参数量为 (考虑bias)

$$\begin{aligned} \theta_single &= d_model \times d_model + d_model \\ &= d_model^2 + d_model \\ &= d^2 + d \end{aligned} \tag{2}$$

  因此，一个完整的Multi-Head Attention模块的参数量为：

$$\begin{aligned} \theta_MultiHead &= 4 \times (d_model^2 + d_model) \\ &= 4 \times d_model^2 + 4 \times d_model \\ &= 4d^2 + 4d \end{aligned} \tag{3}$$

1.2.3 MLP模型参数（FeedForward层）

- Transformer中的MLP（前馈神经网络）通常包含两个全连接层,结束输入维度为d_model, 中间隐藏层维度为hidden_size(通常为4倍d_model), 输出层维度为d_model,则
  1. 第一层参数量：d_model → hidden_size（通常hidden_size = 4 × d_model，这里考虑bias）

$$
\begin{aligned}
\theta_MLP1 &= d_model \cdot hidden_size + hidden_size \\
&= d_model \cdot 4 \cdot d_model + 4 \cdot d_model \\
&= 4 \cdot d_model^2 + 4 \cdot d_model \\
&= 4d^2 + 4d
\end{aligned} \tag{4}
$$

2. 第二层参数量：hidden_size → d_model

$$
\begin{aligned}
\theta_MLP2 &= hidden_size \cdot d_model + d_model \\
&= 4 \cdot d_model \cdot d_model + d_model \\
&= 4 \cdot d_model^2 + d_model \\
&= 4d^2 + d
\end{aligned} \tag{5}
$$

需要注意的是，第一层bias参数量为 hidden_size = 4d_model,第二层bias参数量为d_model

MLP总参数量

$$
\begin{aligned}
\theta_MLP &= \theta_MLP1 + \theta_MLP2 \\
&= 4 \cdot d_model^2 + 4 \cdot d_model + 4 \cdot d_model^2 + d_model \\
&= 8 \cdot d_model^2 + 5 \cdot d_model \\
&= 8d^2 + 5d
\end{aligned} \tag{6}
$$

1.2.4 Layer Normalization参数

- Layer Normalization包含两组参数：缩放因子 γ 和偏置 β ，每个参数的维度等于特征维度d_model。

  每个Layer Norm的参数量为$2 \cdot d_model$, 而在ransformer中，每个编码器/解码器层通常有两个Layer Norm（一个在Self-Attention后，一个在MLP后），因此每层的Layer Norm参数量为：

$$
\theta_LayerNorm = 4 \cdot d_model = 4d \tag{7}
$$

1.2.5 TransformerBlock参数

- 每个Transformer块的参数为 (注意这里是Transformer块，不包含Embedding层和最后的Linear层)

$$
\begin{aligned}
\theta_TransBlock &= \theta_MultiHead + \theta_MLP + \theta_LayerNorm \\
&= 12 \cdot d_model^2 + 13 \cdot d_model \\
&\approx 12 \cdot d
\end{aligned} \tag{8}
$$

1.2.6 Linear层

- 最后的Linear层作用是将TransformerBlock的输出映射到词汇表大小,假设词汇表大小为vocab_size, 模型维度为d_model

$$\begin{aligned}\theta_Linear &= vocab_size \cdot d_model \\ &= v \cdot d\end{aligned} \tag{9}$$

1.3 示例-LLaMA模型参数量

- LLaMA模型介绍

  LLaMA-1是Meta AI在2023年2月发布的一系列大型语言模型。LLaMA-1模型有四种不同规模的版本

  模型架构特点

  1. **词汇表大小**：LLaMA-1使用的词汇表大小为32,000个token。
  2. **主要组件参数量**：
     - **Embedding层**：vocab_size × d_model
       - 例如7B模型：32,000 × 4,096 ≈ 131M参数
     - **每个Transformer层**：
       - Self-Attention: 4 × d_model^2
       - MLP: 8 × d_model^2 (使用SwiGLU激活函数)
       - Layer Normalization: 4 × d_model
     - **7B模型每层参数量**：
       - Self-Attention: 4 × $4{,}096^2$ ≈ 67.1M
       - MLP: 8 × $4{,}096^2$ ≈ 134.2M
       - Layer Norm: 4 × 4,096 ≈ 0.016M
       - 每层总计: ≈ 201.3M
       - 32层总参数量: 32 × 201.3M ≈ 6.44B
  3. **特殊优化**：
     - 使用了RMSNorm进行归一化
     - 采用了旋转位置编码(RoPE)
     - 使用SwiGLU激活函数替代ReLU
     - 去除了绝对位置编码
  4. **训练数据**：LLaMA-1模型在约1.4万亿tokens的数据上进行了训练

  与GPT-3相比，LLaMA-1在相同参数量级别下，通过更高效的架构设计和更多的训练数据，实现了相当或更好的性能。例如，LLaMA-13B的性能可以与GPT-3（175B）相媲美，而参数量只有后者的约7.5%。

- 以LLaMA模型为例，我们可以估算其参数量。LLaMA有多个版本，以LLaMA-1为例：

  注意这里没有加上embedding层和最终归一化层

模型版本	参数量	隐藏层维度(d_model)	层数	$12 \cdot l \cdot d_model^2$
LLaMA-7B	6.7B	4,096	32	6，442，450，944
LLaMA-13B	13.0B	5,120	40	12，582，912，000
LLaMA-33B	32.5B	6,656	60	31，897，681，920
LLaMA-65B	65.2B	8,192	80	64，424，509，440

1.4 常见操作的FLOPs计算：

1.4.1 Floating Point Operation简介

- 浮点运算（Floating Point Operation，简称FLOP）是指计算机进行的浮点数数学运算。在深度学习中，我们通常使用FLOPs（Floating Point Operations Per Second）来衡量计算速度，使用总FLOPs来衡量计算量。
- 一个FLOP代表一次浮点运算，包括：加法操作、减法操作、乘法操作、除法操作以及平方根等特殊函数操作
  在深度学习领域，通常将每次基本的数学运算（如加、减、乘、除）都计为1个FLOP。
- 需要注意的是，FLOPs（浮点运算次数）与FLOPs/s（每秒浮点运算次数）是两个不同的概念：
  1. FLOPs ： 表示完成一个计算任务所需的浮点运算总数，是一个绝对数值，用于衡量算法或模型的计算复杂度。
  2. FLOPs/s （或FLOPS，全大写）： 表示计算设备每秒能执行的浮点运算次数，是衡量计算设备性能的指标。通常以GFLOPS（10^9 FLOPs/s）或TFLOPS（10^12 FLOPs/s）为单位。

1.4.2 向量点积：

- 向量点积是最常见的向量乘法运算，定义为：

$$x \cdot y = \sum_{i=1}^{m} x_i y_i = x_1 y_1 + x_2 y_2 + \ldots + x_m y_m \tag{10}$$

  计算点积涉及的FLOPs数量为：
  1. 每对元素相乘：m次乘法操作
  2. 将所有乘积相加：(m-1)次加法操作
- 因此，两个长度为m的向量点积所需要的FLOPs数约等于

$$2mFLOPs \tag{11}$$

1.4.3 矩阵乘法

- 矩阵乘法，当两个矩阵A[m,k]和B[k,n]相乘时，结果是一个m×n的矩阵C。

  矩阵C中的每个元素C[i,j]的计算公式为：

$$C[i,j] = \sum_{p=1}^{k} A[i,p] \times B[p,j] \tag{12}$$

- 上述公式意味着C矩阵的每个元素都是A矩阵的一行与B矩阵的一列的点积。是不是很熟悉，其实就是没个元素都是两个长度为k的向量点积

  对于C矩阵中的每个元素C[i,j]，我们需要执行以下操作：

  1. **乘法操作**：k次乘法（A[i,1]×B[1,j], A[i,2]×B[2,j], ..., A[i,k]×B[k,j]）
  2. **加法操作**：(k-1)次加法（将k个乘积相加需要k-1次加法操作）

  因此，计算C[i,j]一个元素需要的FLOPs数为：

$$k + (k-1) = 2k - 1 \tag{13}$$

- 由于C矩阵有m×n个元素，每个元素需要(2k-1)个FLOPs，所以总的FLOPs数为：

$$m \times n \times (2k-1) \tag{14}$$

- 简约表示，当k非常大时 $2k-1 \approx$2k ,所以可简约表示为

$$2 \times m \times n \times kFLOPs \tag{15}$$

1.4.4 矩阵加法

- 矩阵加法 (A[m,n] + B[m,n]): m × n FLOPs

  矩阵加法，当两个矩阵A[m,n]和B[m,n]相加时，结果是一个m×n的矩阵C。

  矩阵C中的每个元素C[i,j]的计算公式为：

$$C[i,j] = A[i,j] + B[i,j] \tag{16}$$

- 上述公式意味着C矩阵的每个元素都是A矩阵和B矩阵对应位置元素的简单加法。

  对于C矩阵中的每个元素C[i,j]，我们需要执行以下操作：

  加法操作：1次加法（A[i,j] + B[i,j]）

  因此，计算C[i,j]一个元素需要的FLOPs数为：

$$1FLOPs \tag{17}$$

  由于C矩阵有m×n个元素，每个元素需要1个FLOPs，所以总的FLOPs数为：

$$m \times nFLOPs \tag{18}$$

1.5 计算量估计

1.5.1 MultiHead self-Attention

- 假设数据数据的形式为[batch_size, seq_len],对于Self-attention模块（包含QKV投影矩阵和输出投影矩阵）：

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V \tag{19}$$

$$x_{out} = \mathbf{softmax}(\frac{QK^T}{\sqrt{d_model}}) \cdot V \quad \cdot W_O + x \tag{20}$$

其中：

1. Q、K、V：QKV投影矩阵
2. W_O ：输出投影矩阵的权重
3. 后面加上一个x是残差连接

如 $Q = xW_Q$, x 维度为 [b,s,d] ,W_Q 维度为[d,d],则按照矩阵乘法的FLOPs计算有

$$b \cdot s \cdot d \cdot (2d - 1) \approx 2bsd^2 \tag{21}$$

其他计算类似

	FLOPs
计算Q,K,V	$3 \cdot 2bsd^2$
计算QK^T	$2bs^2d$
计算$softmax(.)@V$	$2bs^2d$
计算输出投影	$2bsd^2$

1.5.2 MLP模块

- 假设数据的形式为[batch_size, seq_len],对于MLP模块,假设隐藏层hidden_size等于4倍d_model

$$x = f_{relu}(x_{out}W_1)W2 + x_{out} \tag{22}$$

继续使用b替代batch_size, s替代seq_len, d替代d_model

如第一层的MLP，是将x_out从 [batch_size,seq_len,d_model] --> [batch_size, seq_len , hidden_size],所以FLOPs等于

$$
\begin{aligned}
FLOPs &= batch_size \cdot seq_len \cdot hidden_size \cdot (2 \cdot d_model - 1) \\
&\approx b \cdot s \cdot 4d \cdot 2d \\
&\approx 8bsd^2
\end{aligned} \tag{23}
$$

其他计算类似

	FLOPs
第一层MLP	$8bsd^2$
第二层MLP	$8bsd^2$

1.5.3 Linear层

- 假设数据的形式为[batch_size, seq_len]，对于最后的Linear层，是将输入映射到词汇表大小，即

  [batch_szie, seq_len, d_model] --> [batch_size,seq_len,vocab_size]

$$
\begin{aligned}
FlOPs &= batch_size \cdot seq_len * vocab_size * (2d_model - 1) \\
&= bsv \cdot (2d - 1) \\
&\approx 2bsvd
\end{aligned} \tag{24}
$$

1.5.4 一个batch的计算量

- `对于其中一个[batch_size, seq_len]`，其计算量self-attention模块 + MLP模块 + Linear层

$$
total_FLOPs = l \cdot (24bsd^2 + 4bs^2d) + 2bsvd \tag{25}
$$

1.5.5 计算量与参数量之间的关系

- 根据我们前面我们计算的参数量，transform blok为 l 的参数量为

$$
12 \cdot d^2 \cdot l \tag{26}
$$

- 对于1个[batch_szie，seq_len]的数据，计算量约等于

$$
l \cdot 24bsd^2 \tag{27}
$$

- 在forward过程中

$$
\frac{l \cdot 24bsd^2}{12 \cdot d^2 \cdot l \cdot b \cdot s} = \frac{FLOPs}{参数量 \cdot Token数} = 2 \tag{28}
$$

其中 $token数 = batch_size * seq_len$

对于1个token，没个模型参数需要进行2个float运算

- 如果forward过程需要1个单位的计算，那么backward大约需要2个单位的计算，中间变量大约需要1个单位计算，总共需要4个单位
- 所以Forward,Backward,中间变量，`对于一个token,每个模型参数需要进行8个Float运算`。

1.5.6 训练时长

- 训练时长计算公式如下

$$
\begin{aligned}
\text{训练时间} &= \frac{\text{计算量}}{GPU\text{数} \times GPU\text{峰值}flops \times GPU\text{利用率}} \\
&= \frac{8 \times tokens \times \text{参数量}}{GPU\text{数} \times GPU\text{峰值}flops \times GPU\text{利用率}}
\end{aligned} \tag{29}
$$